Project Plan

AI-Powered Infrastructure Monitoring with Kubernetes and Cilium

Research

Bas Smit

Fontys University of Applied Sciences

Fontys
University of Applied Sciences

Contents

1 Introduction **1**
1.1 Background and Motivation . . . 1
1.2 Project Scope . . . 1
1.3 Relevance to Enterprise Infrastructure . . . 2

2 Research Questions and Project Objectives **3**
2.1 Main Research Question . . . 3
2.2 Sub-Questions . . . 3

3 Methodology and Research Framework **5**
3.1 Library Research . . . 5
3.1.1 Literature Study . . . 5
3.1.2 Available Product Analysis . . . 5
3.1.3 Best, Good, and Bad Practices . . . 5
3.2 Field Research . . . 6
3.2.1 Problem Analysis . . . 6
3.2.2 Domain Modelling . . . 6
3.3 Workshop Research . . . 6
3.3.1 IT Architecture Sketching . . . 6
3.3.2 Prototyping . . . 6
3.4 Lab Research . . . 6
3.4.1 System Test . . . 7
3.4.2 Security Test . . . 7
3.4.3 Non-Functional Test . . . 7
3.5 Showroom Research . . . 7
3.5.1 Benchmark Test . . . 7
3.5.2 Guideline Conformity Analysis . . . 7
3.6 Framework Triangulation . . . 7
3.7 Additional Frameworks . . . 7

4 Non-Functional Requirements **9**
4.1 Scalability . . . 9
4.2 Performance . . . 9
4.3 Security . . . 9
4.4 Reliability . . . 10
4.5 Observability . . . 10
4.6 Maintainability and Portability . . . 10

5 Planning, Timeline, and Expected Challenges **11**
5.1 Project Timeline . . . 11
5.2 Time Estimation . . . 12
5.3 Expected Deliverables . . . 13
5.4 Expected Challenges . . . 13
5.4.1 Training Data for AI Anomaly Detection . . . 13
5.4.2 eBPF Kernel Version Compatibility . . . 13
5.4.3 Integration Complexity . . . 13
5.4.4 Scope Creep . . . 14
5.5 Supervisor Approval . . . 14

1 Introduction

1.1. Background and Motivation

Enterprise infrastructure has grown substantially more complex over the last decade. Kubernetes has become the de-facto standard for container orchestration, but its adoption introduces two interrelated and persistent challenges: *reactive monitoring* and *insufficient network security visibility*.

Traditional threshold-based monitoring systems are designed for static, predictable workloads. In a dynamic Kubernetes environment — where pods are ephemeral, traffic patterns shift continuously, and dozens of microservices interact — fixed alert thresholds produce excessive false positives and frequently miss subtle, multi-dimensional anomalies that precede real outages. The result is alert fatigue for on-call engineers and a fundamentally *reactive* posture: problems are discovered only after they have already impacted services.

On the security side, the default Kubernetes networking model relies on `iptables` rules managed by `kube-proxy`. This approach operates exclusively at Layer 3/4 (IP and port), has limited observability, and becomes a performance bottleneck at scale. It offers no visibility into the content or identity of application-layer traffic and provides no mechanism for detecting lateral movement or anomalous flow behaviour within the cluster.

Cilium[1] addresses these networking limitations through the use of eBPF (extended Berkeley Packet Filter) — a kernel-level technology that allows safe, sandboxed programs to run inside the Linux kernel without modifying kernel source code. Cilium enforces network policies at Layer 3 through Layer 7 (including HTTP, gRPC, and Kafka), provides mutual TLS encryption between workloads, and exposes rich, real-time network flow telemetry through its Hubble observability layer.

Critically, the combination of AI-driven monitoring and Cilium-based network security is deeply complementary. Cilium's Hubble component generates granular per-flow telemetry that can serve as a high-value input to machine-learning anomaly detection models. Conversely, AI-detected anomalies in infrastructure metrics can trigger dynamic responses at the network layer. Together these technologies enable an infrastructure that is *self-aware* and *self-defending*: capable of detecting emerging problems proactively and responding to threats autonomously.

1.2. Project Scope

This project investigates how an AI-powered monitoring system and a Cilium-secured Kubernetes network can be designed, integrated, and validated to provide proactive, automated infrastructure management in an enterprise context.

The scope covers:

- A study of current Kubernetes monitoring and networking limitations.

[1] `https://cilium.io`

- The design of an integrated architecture combining Prometheus, an AI anomaly detection engine, Cilium, and Hubble.
- A proof-of-concept implementation deployed on a bare-metal Kubernetes cluster.
- Empirical evaluation of the solution's effectiveness and performance overhead.

1.3. Relevance to Enterprise Infrastructure

Enterprise infrastructure teams are under growing pressure to maintain service reliability while managing increasingly complex, distributed systems. Proactive anomaly detection reduces mean-time-to-detect (MTTD) and mean-time-to-resolve (MTTR) for incidents, directly translating to higher service availability. The shift to zero-trust networking — mandated by frameworks such as NIST SP 800-207 and increasingly required by enterprise security policies — demands the kind of identity-aware, application-layer network policy enforcement that Cilium provides.

By combining these two capabilities, this project addresses a genuine gap in the current enterprise tooling landscape and produces a reusable, open-source-based blueprint that infrastructure teams can adopt.

2

Research Questions and Project Objectives

2.1. Main Research Question

How can an AI-powered monitoring system, integrated with Cilium-based network security, be designed and implemented to enable proactive and automated management of a Kubernetes infrastructure?

2.2. Sub-Questions

The main question is decomposed into five sub-questions, each addressing a distinct aspect of the investigation:

1. **What are the key limitations of traditional infrastructure monitoring and network security approaches in Kubernetes environments?**

   This sub-question establishes the problem baseline. It examines where threshold-based alerting fails to detect real anomalies, how `kube-proxy/iptables` creates security blind spots, and what observability gaps exist in a default Kubernetes deployment.

2. **Which AI and machine learning techniques are best suited for anomaly detection and predictive alerting on infrastructure and network metrics?**

   This sub-question surveys the machine learning literature for methods applicable to time-series infrastructure data (e.g. Isolation Forest, LSTM autoencoders, Prophet, ARIMA). It evaluates them against criteria including latency requirements, interpretability, and the availability of labelled training data.

3. **How does Cilium's eBPF-based architecture improve network security and observability compared to traditional Kubernetes networking solutions?**

   This sub-question performs a comparative analysis of Cilium against standard Kubernetes networking (Flannel, Calico, kube-proxy), with a focus on policy expressiveness (L3–L7), performance overhead, observability depth via Hubble, and zero-trust capabilities such as transparent mutual TLS.

4. **What integration architecture is required to enable the AI monitoring system to consume Cilium and Hubble telemetry for correlated security and performance insights?**

   This sub-question addresses the design challenge of unifying diverse telemetry streams — Prometheus metrics, Loki logs, OpenTelemetry traces, and Hubble network flows — into a coherent feature space for machine learning inference, and feeding alerts back into actionable responses.

5. **How can the effectiveness, performance overhead, and security posture of the integrated solution be empirically validated?**

   This sub-question defines the evaluation methodology: what metrics are used to measure proactive detection quality (precision, recall, MTTD), how eBPF performance overhead is quantified, and how Cilium network-policy coverage is assessed against the CIS Kubernetes Benchmark.

3

Methodology and Research Framework

This project applies the *Development Oriented Triangulation* (DOT) framework[1] to structure the research. The DOT framework distinguishes three research levels: *what* is researched (domains), *why* it is researched (trade-offs between fit, expertise, overview, and certainty), and *how* it is researched (strategies and methods). The sections below map each research strategy to the specific methods and activities carried out in this project.

3.1. Library Research

Library research explores what is already known and what tools, theories, and guidelines can inform the project. It primarily targets the *available work* domain and optimises for *expertise*.

3.1.1. Literature Study

A systematic literature study will be conducted covering:

- AI/ML methods for time-series anomaly detection (Isolation Forest, LSTM autoencoders, Prophet, ARIMA, and transformer-based approaches).
- eBPF internals, the Cilium architecture, and the Hubble observability layer.
- Zero-trust network models (NIST SP 800-207) and their application to Kubernetes.
- The CNCF landscape for observability and security tooling.

This method primarily answers sub-questions 1, 2, and 3.

3.1.2. Available Product Analysis

A comparative analysis of existing tools will be performed across two categories:

- **Monitoring stack:** Prometheus, VictoriaMetrics, Grafana, Loki, OpenTelemetry Collector, Jaeger, and commercial alternatives (Datadog, Dynatrace, New Relic).
- **Security and networking:** Cilium, Calico, Flannel, Falco, Tetragon, and Hubble.

Each tool is assessed on criteria including open-source licence, Kubernetes-native support, community maturity, feature coverage, and integration feasibility.

3.1.3. Best, Good, and Bad Practices

The following industry standards and guidelines will be reviewed:

[1] `https://ictresearchmethods.nl/dot-framework/`

- CIS Kubernetes Benchmark (v1.9)
- CNCF Cloud Native Security Whitepaper
- NIST SP 800-207 (Zero Trust Architecture)
- OWASP Kubernetes Security Cheat Sheet

3.2. Field Research

Field research explores the *application domain* and optimises for *fit* between the solution and its operational context.

3.2.1. Problem Analysis

A structured problem analysis will characterise the monitoring and security gaps in a representative, freshly deployed Kubernetes cluster (kubeadm, three worker nodes). The analysis will document:

- Alerts fired per day using only threshold-based rules.
- Network flows that are invisible to the default networking stack.
- Incidents that would go undetected without AI-based detection.

This method directly answers sub-question 1.

3.2.2. Domain Modelling

An ArchiMate domain model will be produced to represent the enterprise Kubernetes infrastructure: nodes, namespaces, deployments, services, ingress controllers, storage classes, and external dependencies. This model provides the shared vocabulary for the architecture design phase.

3.3. Workshop Research

Workshop research explores opportunities and produces concrete design artefacts.

3.3.1. IT Architecture Sketching

Using ArchiMate 3.2 notation, the following architecture views will be produced:

- **Motivation view:** Goals, requirements, and constraints.
- **Application layer view:** Monitoring components and their data flows.
- **Technology layer view:** Kubernetes objects, Cilium, and the AI inference pipeline.
- **Implementation and migration view:** Deployment phases.

3.3.2. Prototyping

An iterative prototyping approach will be used to implement and refine each major component:

1. Cilium installation and basic NetworkPolicy enforcement.
2. Hubble Relay and Hubble UI for flow visualisation.
3. Prometheus + Grafana stack with custom Kubernetes dashboards.
4. AI anomaly detection pipeline (training on synthetic normal-traffic baseline, inference via REST API).
5. Integration layer: Hubble flow exporter → OpenTelemetry Collector → AI feature store.

3.4. Lab Research

Lab research tests whether the solution works as intended. It optimises for *certainty*.

3.4.1. System Test

An end-to-end system test will verify that the full monitoring and security stack operates correctly: metrics are scraped, network flows are captured, anomaly detection fires on injected anomalies, and alerts reach the notification channel.

3.4.2. Security Test

Cilium network policies will be validated by attempting traffic flows that should be blocked (e.g. cross-namespace access without an explicit allow policy, egress to an unauthorised external host). Tetragon will be used to verify syscall-level enforcement.

3.4.3. Non-Functional Test

The following non-functional properties will be measured:

- CPU and memory overhead introduced by Cilium/eBPF relative to a Flannel baseline.
- Monitoring pipeline latency (time from metric generation to alert firing).
- AI inference throughput and model prediction latency under load.

3.5. Showroom Research

3.5.1. Benchmark Test

The AI anomaly detection component will be benchmarked against a rule-based baseline using precision, recall, F1-score, false-positive rate, and MTTD (mean time to detect). Anomaly scenarios will be injected in a controlled test environment.

3.5.2. Guideline Conformity Analysis

The final implementation will be assessed against the CIS Kubernetes Benchmark and the CNCF Security Whitepaper recommendations. Findings will be reported as a conformity scorecard in the final research report.

3.6. Framework Triangulation

Table 3.1 summarises the triangulation across research strategies used to answer each sub-question.

Table 3.1: DOT Framework triangulation: research methods per sub-question.

Sub-question	Primary methods
SQ1 — Limitations of current approach	Problem analysis (Field), Literature study (Library)
SQ2 — AI/ML technique selection	Literature study (Library), Available product analysis (Library)
SQ3 — Cilium vs. traditional networking	Literature study (Library), Benchmark test (Showroom)
SQ4 — Integration architecture	IT architecture sketching (Workshop), Prototyping (Workshop)
SQ5 — Validation	System test (Lab), Security test (Lab), Non-functional test (Lab), Guideline conformity (Showroom)

3.7. Additional Frameworks

Beyond the DOT framework, the following methodological frameworks are applied:

- **TOGAF ADM (Architecture Development Method):** The architecture design phase follows the TOGAF ADM cycle, moving from Architecture Vision through Business, Application, and Technology Architecture to the Opportunities and Solutions phase.

- **ArchiMate 3.2:** Used as the modelling notation for all architecture views, ensuring interoperability and a standardised vocabulary.
- **GitOps (Flux CD):** All infrastructure-as-code artefacts are managed in a Git repository and reconciled to the cluster automatically, ensuring reproducibility of the proof-of-concept environment.

4

Non-Functional Requirements

The following non-functional requirements (NFRs) constrain the design and implementation of the integrated solution. They are derived from enterprise infrastructure best practices, the reviewed standards (CIS Kubernetes Benchmark, NIST SP 800-207), and the project context.

4.1. Scalability

- **NFR-SC-01:** The monitoring stack must remain operational and responsive on clusters of up to 100 worker nodes and 500 running pods without manual reconfiguration.
- **NFR-SC-02:** The AI inference pipeline must scale horizontally; throughput must increase linearly when additional inference replicas are added.
- **NFR-SC-03:** Cilium must operate at wire speed on 10 Gbps network interfaces without packet drops under normal cluster load.

4.2. Performance

- **NFR-PE-01:** The additional CPU overhead introduced by Cilium (eBPF programs) must not exceed 5% of total cluster CPU capacity compared to a Flannel baseline under equivalent workloads.
- **NFR-PE-02:** The additional memory footprint of the Cilium agent must not exceed 200 MiB per node.
- **NFR-PE-03:** The end-to-end monitoring latency (metric generation to alert notification) must be below 60 seconds for critical severity alerts.
- **NFR-PE-04:** AI model inference latency for a single prediction must be below 500 ms at the 99th percentile.

4.3. Security

- **NFR-SE-01:** All inter-service communication within the cluster must be governed by explicit CiliumNetworkPolicy resources; a default-deny posture must be enforced at both ingress and egress.
- **NFR-SE-02:** Mutual TLS (mTLS) must be enforced transparently between all workloads using Cilium's WireGuard or IPsec encryption, without requiring application code changes.
- **NFR-SE-03:** The solution must achieve a passing score on the applicable controls of the CIS Kubernetes Benchmark v1.9.
- **NFR-SE-04:** All container images used in the monitoring and security stack must be scanned for known CVEs prior to deployment; images with critical vulnerabilities must not be deployed.

- **NFR-SE-05:** Secrets (API keys, certificates, credentials) must not be stored in plaintext; they must be managed via Kubernetes Secrets with RBAC access controls or an external secrets manager.

4.4. Reliability

- **NFR-RE-01:** The monitoring stack (Prometheus, Grafana, AlertManager) must be deployed in high-availability mode with a minimum of two replicas per component and anti-affinity rules to prevent co-location on the same node.
- **NFR-RE-02:** Metric data retention must be a minimum of 30 days, with long-term storage backed by persistent volumes.
- **NFR-RE-03:** The Cilium agent must recover automatically from pod restarts without manual intervention, re-establishing all enforced network policies within 30 seconds.
- **NFR-RE-04:** The AI anomaly detection service must be resilient to the loss of a single input data stream; it must continue to produce predictions using available data and emit a degraded-mode alert.

4.5. Observability

- **NFR-OB-01:** Full L3–L7 network flow visibility must be provided via Hubble for all intra-cluster and cluster-egress traffic.
- **NFR-OB-02:** The solution must support the three pillars of observability: metrics (Prometheus), logs (Loki), and distributed traces (OpenTelemetry/Jaeger), with correlation across all three signals.
- **NFR-OB-03:** Grafana dashboards must provide at minimum: cluster-level resource utilisation, per-namespace network flow heatmaps, anomaly detection score time series, and active network policy violation counts.

4.6. Maintainability and Portability

- **NFR-MA-01:** All components must be deployed exclusively via Helm charts stored in a version-controlled Git repository; no manual `kubectl apply` steps should be required after initial cluster bootstrap.
- **NFR-MA-02:** The GitOps reconciliation loop (Flux CD) must automatically converge the cluster state to the desired state defined in Git within 5 minutes of a commit.
- **NFR-MA-03:** The solution must be validated on at least two distinct Kubernetes distributions: bare-metal (kubeadm) and a managed cloud service (AKS or EKS).

5

Planning, Timeline, and Expected Challenges

5.1. Project Timeline

The remaining project period runs from 12 March 2026 to 28 June 2026. Working two days per week (09:00–16:00), this yields approximately 15 working weeks. Table 5.1 maps each phase to a concrete date range.

Table 5.1: Project timeline.

Phase	Dates	Activities	Deliverable
Project Plan	12 Mar	Finalise and submit plan of approach; supervisor review and approval	**This document**
Library Research	19 Mar – 2 Apr	Systematic literature study (AI/ML anomaly detection, eBPF, Cilium, zero-trust); available product analysis	Literature review
Architecture Design	2 Apr – 16 Apr	Problem analysis on reference cluster; domain modelling; ArchiMate architecture views (TOGAF ADM phases A–C)	Architecture document
Implementation — Cilium	16 Apr – 7 May	Install Cilium; configure CiliumNetworkPolicy (default-deny); enable WireGuard mTLS; deploy Hubble Relay and UI; validate L3–L7 policy enforcement	Cilium PoC
Implementation — AI Monitoring	7 May – 21 May	Deploy Prometheus + Loki + OpenTelemetry stack; build AI feature pipeline (Hubble → OTel Collector → feature store); train anomaly detection model; expose inference API	AI monitoring pipeline
Integration	21 May – 4 Jun	Integrate AI alerts with AlertManager; wire Hubble flow data into ML features; end-to-end smoke tests	Integrated PoC
Testing & Evaluation	4 Jun – 18 Jun	System tests; security tests (policy violation injection); non-functional tests (CPU/memory overhead); benchmark vs. threshold-based baseline	Test & benchmark report
Report Writing	18 Jun – 28 Jun	Draft final research report; peer review; incorporate feedback; final submission	**Final report**

5.2. Time Estimation

Available capacity: **2 days per week**, **09:00–16:00** (7 hours per day), from 12 March 2026 to 28 June 2026.

$$15 \text{ weeks} \times 2 \text{ days} \times 7 \text{ hours} = \textbf{210 hours total}$$

Table 5.2 shows the estimated effort distribution per phase.

Table 5.2: Estimated effort distribution.

Phase	Working days	Hours
Project Plan	1	7
Library Research	4	28
Architecture Design	4	28
Implementation — Cilium	5	35
Implementation — AI Monitoring	4	28
Integration	3	21
Testing & Evaluation	4	28
Report Writing	3	21
Buffer / contingency	2	14
Total	**30**	**210**

5.3. Expected Deliverables

The following artefacts will be produced over the course of the project:

1. **Project Plan** (12 March) — this document.
2. **Literature Review** (2 April) — a structured review covering AI/ML anomaly detection techniques, Cilium/eBPF architecture, and relevant standards.
3. **Architecture Document** (16 April) — ArchiMate views (motivation, application, and technology layers) following the TOGAF ADM.
4. **Proof-of-Concept Implementation** (4 June) — all infrastructure-as-code (Helm charts, Flux manifests) and the AI pipeline source code, stored in a Git repository.
5. **Test & Benchmark Report** (18 June) — structured test results including CIS Benchmark conformity scorecard, security test outcomes, and performance benchmark comparisons.
6. **Final Research Report** (28 June) — a complete research report answering all research questions and providing actionable recommendations.

5.4. Expected Challenges

5.4.1. Training Data for AI Anomaly Detection

Training an unsupervised anomaly detection model requires a representative baseline of normal infrastructure behaviour. Generating this baseline on a freshly provisioned test cluster means the model must be trained on synthetic or simulated workloads, which may not fully reflect the diversity of production traffic patterns. Mitigation: use an unsupervised technique (Isolation Forest) for the initial model, which does not require labelled anomaly data; supplement with synthetic anomaly injection for evaluation.

5.4.2. eBPF Kernel Version Compatibility

Certain Cilium features (e.g. WireGuard transparent encryption, socket-level load balancing) require a minimum Linux kernel version (5.10+). Bare-metal hardware running older kernels may not support all planned features. Mitigation: validate kernel version requirements early during the Cilium implementation phase (from 16 April); document any feature gaps and use alternative mechanisms (IPsec) where WireGuard is unavailable.

5.4.3. Integration Complexity

Correlating heterogeneous telemetry streams — Prometheus counter metrics, Loki log entries, OpenTelemetry spans, and Hubble flow records — into a unified feature vector for machine learning is non-trivial. Each data source has different cardinality, granularity, and latency characteristics. Mitigation: use the OpenTelemetry Collector as a central pipeline with explicit fan-in transformation; define a minimal feature set for the initial model and expand iteratively.

5.4.4. Scope Creep

The combination of AI/ML and advanced networking touches a broad technology surface. There is a risk of spending too much time on peripheral concerns (e.g. model hyper-parameter tuning, advanced Cilium enterprise features) at the expense of completing the core research. Mitigation: strictly follow the timeline; use timeboxing for each implementation sprint; discuss scope changes with the supervisor before acting on them.

5.5. Supervisor Approval

This plan of approach requires review and written approval by the supervising lecturer before research activities commence on 19 March 2026.

Student:	**Supervisor:**
Bas Smit	Ederveen, Martin M.W.
Date: _____	Date: _____
Signature: _____	Signature: _____